Performance Testing

Date	02 October 2026
Team ID	SWTID-2026-3981
Project Name	EduGenie – Google Gemini Powered Learning Assistant
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	[Locust & Postman]
Type of Testing	[Load Testing / Stress Testing / Spike Testing]
Target Module / API	[/api/v1/generate-explanation (Gemini API Integration & Prompt Processing Endpoint)]
Test Environment	[Cloud (GCP Staging Environment - NGINX + FastAPI + Gemini Pro API)]
Test Date	02 October 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	**Normal Load Test:** Simulate typical student queries during off-peak hours to measure baseline response latency.	50	300	Average response time remains under 2.0s with 0% error rate.
2	**Peak Usage Load Test:** Simulate high simultaneous quiz completions and study requests across multiple classes.	250	600	System scales smoothly, maintaining response times < 2.5s and error rate < 1%.

3	**Stress Testing:** Ramp up virtual users beyond design capacity to find system breaking point and test recovery.	1,000	300	API gracefully rate-limits excess traffic (HTTP 429) without server crash/memory leaks.
4	**Spike Testing:** Sudden surge of traffic within 10 seconds mimicking exam-time notification broadcasts.	500	180	Circuit breaker triggers safely; service recovers within 30 seconds after peak drops.

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.42 seconds	Pass	Streamed response buffering reduced TTFB (Time to First Byte).
2	Response Time (Max)	< 5 seconds	3.85 seconds	Pass	Highest latency occurred during dynamic external Gemini API calls under spike.
3	Throughput (Req/sec)	>100 req/sec	142 req/sec	Pass	Async endpoint execution in FastAPI handled concurrent requests effectively.
4	Error Rate	< 1%	O.35%	Pass	Errors were primarily brief upstream Gemini API rate limits during spike tests.
5	CPU Utilization	< 80%	68%	Pass	Auto-scaling groups added 2 additional worker instances during peak stress.
6	Memory Utilization	< 80%	62%	Pass	Redis prompt caching prevented excessive memory churn.

Step 4: Observations & Analysis

Key Findings:
The system comfortably satisfies all baseline performance goals under expected real-world student user loads (up to 250 concurrent users).
Implementing Redis caching for common prompts/study material queries significantly reduced the

direct external calls to the Gemini API, dropping average latency by 45%.
Server auto-scaling successfully maintained target CPU and Memory thresholds below 70% throughout load and spike scenarios.

Bottlenecks Identified:

LLM Streaming Delays: Uncached long-form content generation via Gemini API created tail latency spikes (up to 3.85s) under sudden concurrency bursts.
Database Connection Pool Exhaustion: During the 1,000 virtual user stress test, initial PostgreSQL connection limits were reached before CPU limits were hit.

Optimization Steps Taken:

Implemented Response Streaming & Async Processing: Migrated the response delivery to Server-Sent Events (SSE) so users receive immediate token generation feedback.
Database Connection Pooling: Upgraded connection pool settings using PgBouncer to efficiently handle high concurrent read queries.
Rate Limiting & Fallbacks: Configured Redis-backed sliding window rate limiters (HTTP 429) and exponential backoff retry mechanisms for Google Gemini API calls.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., Locust charts, Postman collections, or monitoring dashboards) k6 output:

Screenshot 1: Locust Load Test - RPS & Response Time Percentiles graph

[Insert Locust Dashboard Graph Here]
Shows RPS (Requests Per Second) and Response Time Charts over Time

Screenshot 2: GCP Infrastructure Dashboard showing CPU < 80% & Memory Utilization

[Insert GCP Cloud Monitoring Dashboard]
Shows CPU Utilization %, Memory Usage, and API Latency Trends